Differential Flatness Pursuit and Motor Voltage Dynamics

Md. Shakib Al Sifat
Student ID: 202613880
King Fahd University of Petroleum & Minerals

28 October 2026

Evidence status. Software results labelled SIMULATION are validation data from an ideal kinematic plant. Results labelled HARDWARE come from the physical RoboMaster EP. Model voltages are estimates from the assessment equation, not direct voltmeter measurements.

1 Objective and implementation environment

The task is to search for AprilTag 3, align in place, follow a cubic trajectory, and stop at a nominal 0.20 m standoff. The implementation uses Ubuntu 24.04 in WSL 2, ROS 2 Jazzy, Python 3.12, NumPy, OpenCV, and the RoboMaster SDK. Windows connects directly to the robot's Wi-Fi access point. WSL uses mirrored networking; ROS nodes communicate locally in domain 42. A successful local simulation does not establish robot connectivity or physical stopping accuracy.

The ROS package `rm_flatness_nav` contains a planner, an SDK driver, a voltage logger, and an ideal kinematic simulator. The driver starts with chassis motion disarmed, initializes the arm, receives ESC and ToF data, and decodes the camera stream. All internal motion calculations use metres, seconds, and radians, with body x forward and positive yaw counterclockwise. SDK yaw commands are converted to clockwise degrees per second.

2 Visual acquisition and supervisory control

2.1 Arm pose and camera geometry

The driver requests the adjusted camera pose $(X, Y) = (105, 60)$ mm. Raising the second arm coordinate brought the floor tag into the camera view while the chassis remained stationary. The intended arm placement leaves the front ToF beam unobstructed. These coordinates alone do not prove optical pitch or beam clearance: the actual camera bracket, linkage geometry, and sensor location must be inspected on the robot. A stationary near/far ToF test and a camera image at this pose are required evidence.

The printed marker is AprilTag family `36h11`, ID 3, with a measured 120 mm outer black square. Detection uses calibrated intrinsics and square pose estimation. Optical-frame coordinates are transformed to the chosen body reference using a measured rigid transform. The planar distance is $d = \sqrt{x_b^2 + y_b^2}$, and bearing is $\theta_b = \operatorname{atan2}(y_b, x_b)$. Using optical-axis depth directly would be incorrect for a downward-facing camera. The precise chassis reference for standoff and its relationship to the front ToF origin are recorded during calibration.

Stationary hardware preflight. WSL mirrored networking selected the robot access-point route at `192.168.2.1`. The SDK delivered sensor callbacks and a valid H.264 camera stream. The original OpenCV background receiver could connect after the H.264 parameter sets and then

stall; replacing it with the SDK incremental decoder restored bounded latest-frame capture. At the adjusted arm pose $(105, 60)$ mm, the camera returned 1280×720 images and detected Tag 3 in 14/14 sampled frames. The front ToF subscription returned four zero slots with zero IDs and flags, indicating that no valid ToF module was registered. Camera range therefore remains the primary stopping guard; a valid ToF reading would still trigger the same priority stop.

2.2 Finite state machine

SEARCHING: With no fresh target observation for more than 0.45 s, command $v = 0$ and $\omega = \pm 0.30$ rad/s toward the last observed bearing. A default positive direction is used before the first observation.

ALIGNING: With a visible target outside $|\theta_b| \leq 0.18$ rad, rotate in place with bounded proportional bearing control. Aligning before translation reduces lateral drift and helps retain the tag in the camera view.

CHASING: Solve the cubic at the aligned starting range and evaluate its analytical derivatives at 20 Hz. Bounded bearing and range feedback compensate small tracking errors. Large bearing errors return to in-place alignment.

ARRIVED: A fresh camera distance or valid front ToF distance at or below 0.20 m commands immediate zero translation and rotation. The stop is latched.

Sensor loss commands a stationary hold; missing commands trigger a driver watchdog. If the planned horizon expires without the standoff guard, the planner records `STOPPED_SHORT`, not success. The SDK timeout is a host timer and is not an independent robot-side emergency stop. A person remains beside the robot during trials.

The camera grab loop includes `time.sleep(0.010)` between iterations, drains the SDK decoder with the `newest` strategy, and retains one application frame. This follows the assessment's buffer-recycling guidance and avoids an unbounded application frame queue. Frame freshness is checked separately from tag visibility.

3 Differential flatness and the cubic solution

For flat output $z = (x, y)^T$, the unicycle model gives

$$v = \sqrt{\dot{x}^2 + \dot{y}^2}, \qquad \omega = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}. \tag{1}$$

At zero speed the angular expression is undefined. The implementation explicitly commands $(v, \omega) = (0, 0)$ when the squared speed is below 10^{-12}, rather than dividing by zero.

For each axis $j \in \{x, y\}$, let $z_j(t) = a_{0j} + a_{1j}t + a_{2j}t^2 + a_{3j}t^3$. The boundary system is

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} a_{0j} \\ a_{1j} \\ a_{2j} \\ a_{3j} \end{bmatrix} = \begin{bmatrix} z_j(0) \\ \dot{z}_j(0) \\ z_j(T) \\ \dot{z}_j(T) \end{bmatrix}. \tag{2}$$

The function `solve_flat_polynomial` calls `numpy.linalg.solve` on this matrix with both axes as right-hand-side columns. Set $D = d_0 - 0.20$, $v_0 = 0.06$ m/s, and

$$T = \max\left(4.0, \frac{1.6D}{0.16}\right). \tag{3}$$

The boundary values are $x(0) = 0$, $\dot{x}(0) = v_0$, $x(T) = D$, $\dot{x}(T) = 0$; all four y boundary values are zero. Thus

$$a_{0x} = 0, \qquad a_{1x} = v_0, \qquad a_{2x} = \frac{3D}{T^2} - \frac{2v_0}{T}, \quad a_{3x} = -\frac{2D}{T^3} + \frac{v_0}{T^2}, \tag{4}$$

$$\dot{x}(t) = a_{1x} + 2a_{2x}t + 3a_{3x}t^2, \quad \ddot{x}(t) = 2a_{2x} + 6a_{3x}t. \tag{5}$$

All y coefficients are zero, so nominal feedforward yaw is zero during the aligned straight pursuit. Nonzero chasing yaw comes from explicitly identified bearing feedback, not from an invented curved y trajectory. For example, $d_0 = 1.65\,\text{m}$ gives $D = 1.45\,\text{m}$ and $T = 14.5\,\text{s}$. The prescribed initial $0.06\,\text{m/s}$ boundary produces a command step from stationary alignment; the cubic has a smooth terminal velocity but does not itself provide a zero-speed launch ramp.

4 Wheel rates and motor voltage model

For $r = 0.05\,\text{m}$ and $L_c = 0.20\,\text{m}$, wheel order is FR, FL, RL, RR:

$$\omega_1 = \omega_4 = \frac{v_x + L_c\omega_z}{r}, \qquad \omega_2 = \omega_3 = \frac{v_x - L_c\omega_z}{r}. \tag{6}$$

The assessment's model is

$$V_i[k] = 0.38\omega_i[k] + 0.06\frac{\omega_i[k] - \omega_i[k-1]}{t_k - t_{k-1}}. \tag{7}$$

These are model-based voltage estimates, not voltage-sensor measurements. The implemented primary series uses acknowledged chassis commands mapped through the prescribed wheel kinematics. Raw ESC speed telemetry is preserved for comparison, with its raw SDK ordering and polarity retained until physically checked. Sampling uses actual monotonic intervals. High-rate execution records preserve transients; a separate table is emitted approximately every second using actual timestamps.

An abrupt stop can yield a nonzero negative acceleration term at the first zero-speed sample. Only subsequent stationary samples necessarily give $0.000\,\text{V}$. For example, changing wheel rate from 2 to $0\,\text{rad/s}$ over $1\,\text{s}$ yields $-0.12\,\text{V}$, followed by zero for two successive stationary samples. The logger never overwrites this braking transient to force a zero result.

5 Validation and experimental results

5.1 Offline software validation

The package was built in a dedicated Ubuntu 24.04 WSL workspace against ROS 2 Jazzy. All 27 unit tests passed. The tests cover the cubic boundary solution, differential-flatness inversion, state transitions, range and ToF stop priority, voltage dynamics, AprilTag pose checks, driver arming gates, command acknowledgement failure, and expiry of stale ToF samples.

A bounded ROS integration simulation exercised the complete node graph. It entered `SEARCHING`, `ALIGNING`, `CHASING`, and `ARRIVED`. The first arrival sample occurred at $t = 16.750\,\text{s}$ with simulated camera range $0.1998\,\text{m}$. Peak forward command was $0.1373\,\text{m/s}$, below the $0.16\,\text{m/s}$ limit. Subsequent stationary model-voltage samples were zero. These results validate software integration only; the simulator does not model wheel slip, motor dynamics, latency, or camera calibration error.

The execution logger now records the cubic position and velocity references, range-derived progress, SDK body velocity, and the associated tracking errors. Range-derived progress is not independent Cartesian odometry, so the report identifies its measurement basis wherever it is plotted.

5.2 Hardware status and pending mission trial

The stationary hardware checks established robot connectivity, live camera transport, decoding, Tag 3 detection, and the adjusted arm pose $(105, 60)$ mm with the chassis held at zero. At that pose Tag 3 was detected in 14/14 sampled frames, with a median image centre of approximately $(636, 203)$ pixels and a median black-square width of 147 pixels. A top-facing calibration candidate at a provisional 0.50 m bumper-to-tag distance gave 0.183 px reprojection RMS; the candidate remains unverified until an independent range check. The ToF interface returned invalid zero values on all channels. The marker must be checked independently at the 1.5–1.8 m start range before chassis arming.

No physical pursuit result is claimed in this revision. Table 1 and the execution plots will be generated from the unique final hardware run after calibration and the supervised low-speed safety test.

Table 1: Chassis motor armature voltage model log (hardware trial pending).

t (s)	V_1 (V)	V_2 (V)	V_3 (V)	V_4 (V)	State
–	–	–	–	–	Awaiting verified hardware run

Video link. https://youtube.com/ADD-FINAL-VIDEO-LINK

6 Reproduction and evidence integrity

Build and test with `scripts/build_test.sh` under WSL. Launch the simulator with `sim:=true`; launch hardware with `sim:=false` and the verified hardware configuration. Trial output directories are unique and CSV files are opened in exclusive-create mode. Simulation rows are marked `SIMULATION`; hardware rows are marked `HARDWARE`. The submission ZIP must contain the ROS package and the actual `motor_voltage_log.csv`. The assessment requires independent student code, hardware trials, and report drafting; this assisted preparation must be reviewed and used consistently with that requirement.

References

[1] A. I. Bhatti, *SCE 672 Term Project Assessment 2: Differential Flatness Pursuit and Motor Voltage Dynamics*, KFUPM, supplied assessment brief, 2026.

[2] DJI, *RoboMaster Developer Guide*, https://robomaster-dev.readthedocs.io/en/latest/python_sdk/robomaster.html.

[3] Microsoft, *Accessing network applications with WSL*, https://learn.microsoft.com/windows/wsl/networking.